A Realistic Backtester for Market Making on BTC/USDT

June 19, 2026

1 Acknowledgement

I would like to formally acknowledge the use of DeepSeek during the process of idea generation, coding, and writing this report.

2 Introduction

I wrote this to ~~have DeepSeek to~~ explain my two Python scripts written for live order book data collection and backtesting with different sets of parameters. The results might be included, but very few are found profitable. I will also document something that I tried and deleted after, and some comments during the process.

The reason why I chose Crypto is that

1. Its data is accessible without charge
2. When I later on invest real money, I can buy 0.00001 BTC, which is affordable as a student

I simply chose Bitcoin because it is the most well-known.

3 Platform for running the scripts

I run the scripts on AWS EC2 24/7 and I chose t4g.small as the model since it is the most cost-effective concerning my needs. Using AWS have the following benefits:

1. I can have my PC resting while my scripts are running online, especially for 24/7 data collection.
2. Using AWS are faster than running the backtesting scripts on my PC, but still it takes several hours to complete.
3. I will not accidentally stop the scripts or kill the terminals.

4 Data Collection (All written by DeepSeek)

All data originates from the Binance exchange. Two complementary data sources are used:

- **Order book snapshots**: a WebSocket stream (‘depth5@100ms‘) provides the top-5 bid and ask price levels with their quantities every 100 milliseconds. These are recorded to daily CSV files, giving the market state at a regular cadence.
- **Trade data**: historical aggregated trades (aggTrades) are downloaded from Binance’s public data portal for the same trading days. Each row corresponds to an executed trade, containing the price, quantity, and a flag ‘is_buyer_maker‘ indicating whether the trade was initiated by a taker buying (i.e., hitting the ask) or selling (hitting the bid).

The WebSocket collector runs on an AWS EC2 instance to ensure 24/7 uptime with minimal latency. The data is stored as CSV files with millisecond-precision timestamps. For backtesting purposes, only the top-of-book (best bid/ask) is required, though deeper levels are recorded for possible future queue modelling.

5 Market Making Model(All written by DeepSeek)

The quoting strategy is based on the Avellaneda–Stoikov (AS) model, which assumes the asset's mid-price S_t follows a Brownian motion with instantaneous variance σ^2. The market maker, who has constant absolute risk aversion γ, maximises the expected utility of terminal wealth. The optimal policy gives:

$$\text{reservation price} = S_t - \gamma\,\sigma^2\,q_t, \tag{1}$$

$$\text{half-spread} = \gamma\,\sigma^2\,(T-t) + \frac{2}{\gamma}\,\ln\!\Big(1+\frac{\gamma}{\kappa}\Big), \tag{2}$$

where q_t is the current inventory (positive when long), $T-t$ is the remaining time, and κ is the order arrival intensity per unit time.

In this implementation, a discretised version is adopted:

- **Volatility estimation**: The variance of log-returns is tracked with an EWMA: $\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)\,r_t^2$, where $r_t = \ln(S_t/S_{t-1})$ and λ is the decay factor. The log-variance is then scaled by S_t^2 to obtain the dollar variance σ_{USDT}^2.
- **Reservation price**: $r = S_t - \gamma\,\sigma_{\text{USDT}}^2\,q_t$.
- **Dynamic spread**: $\delta = \text{base\_half\_spread} + \frac{\gamma}{2}\,\sigma_{\text{USDT}}^2$. The base half-spread is a parameter to be optimised; the AS optimal spread would also depend on κ and time, but the simplified time-independent version already captures the volatility-adaptive behaviour.

Additional features include:

- **Queue simulation**: When a limit order is placed at the best bid/ask price, the size ahead of it (i.e., the resting quantity at that price level) is recorded. A subsequent trade at the same price will only fill our order after all ahead volume has been consumed.
- **Fees**: A maker fee of 0.02% (typical for Binance perpetuals) is applied to every fill.
- **Risk limits**: A hard position limit (2 BTC) and an equity stop-loss (200,000 USDT) terminate the backtest if breached.

6 Backtester Design(All written by DeepSeek)

The backtester processes data in a snapshot-driven, event-by-event loop. At each order-book snapshot, the following steps occur:

1. **Trade processing**: all trades whose timestamp lies between the previous and the current snapshot are consumed. For each trade, the bot's outstanding bid or ask order is checked for a fill, using the queue logic if the price matches exactly.
2. **Order-book crossing**: if the bot's remaining orders now cross the new best bid/ask (i.e., the market has moved), immediate fills are simulated using the snapshot's available volume.
3. **Volatility update & quoting**: the EWMA variance is updated; new bid and ask prices are computed using the inventory-skewed reservation price and dynamic half-spread; a new limit order with fresh queue depths is placed.

4. **Equity marking & risk checks**: the mark-to-market equity is recorded, and the position/equity limits are enforced.

For multi-day experiments, the backtester maintains a state object that carries over position, cash, volatility, the last mid-price (for the first log-return of the next day), and any unfilled order quantity. At each day boundary, the last snapshot of the current day is prepended to the next day's order book, and trades occurring after that snapshot are appended to the next day's trade stream. This ensures continuity without gaps.

The implementation is in Python, relying heavily on pandas for data manipulation and NumPy for numerical operations. The use of 'itertuples' and pre-converted NumPy arrays achieves high performance, enabling a full parameter sweep over 294 combinations per day pair to complete in a few hours on a modest EC2 instance.

7 Experimental Setup(All written by DeepSeek)

The backtest uses two distinct multi-day periods in June 2026:

- Period 1: June 2–4 (June 2 for prior variance, June 3–4 for trading)
- Period 2: June 13–18 (June 13 for prior, June 14–18 for trading)

The prior variance is computed from the first day's entire order book and serves as the initial seed for the EWMA.

The parameter grid explored is:

- reference price: mid
- base half-spread: [0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5] USDT
- risk aversion γ: [0.1, 0.5, 1.0, 1.25, 1.5, 1.75, 2.0]
- EWMA decay λ: [0.99995, 0.9995, 0.995]

Each combination is run for the entire multi-day period, and the final metrics (end equity, position, buy/sell frequency, max/min position, and hit flags) are recorded.

8 Results

TBC

9 Conclusion

TBC

All code is available on GitHub at `https://github.com/KururuZero/Market_Making_Backtester`.

References

[1] Grossman, S.J. and Miller, M.H. (1988). Liquidity and Market Structure. *Journal of Finance.*

[2] Avellaneda, M. and Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance.*

[3] Cartea, Á., Jaimungal, S., and Penalva, J. (2015). *Algorithmic and High-Frequency Trading.* Cambridge University Press.

10 My comments if you want to replicate my work

I have encountered some difficulties or made some serious mistakes. The following are what I think is important. Hope they help OwO

- Probabilistic filling model

  The first time I wrote the backtester, my data is completely faked and the script is very easy. I simply used a random number generator to simulate the price trend movement and whether my order got filled or not. Turn out it was a good head start to test on your logic and determine what variables and parameters you will need what you start using real data (like gamma, half spread, etc)

- Choice of AWS EC2 model

  For free-tier model available, t4g.small is the best one. Especially when you run the backtester, the csv files are very huge in size and need a lot of memory and t3g.small is not enough. Remember the delete the pandas dataframe right after using them to free the memory.

- Be aware of the data source for order book

  If you are using `https://testnet.binance.vision/api` as the API, you are getting fake and sandbox data. To obtain real market data on Binance, use the one in my script. Once I directly copied and pasted from AI and thought it was getting real data, but in fact it wasn't.

- WebSocket Auto-disconnect

  By default, WebSocket will auto discount your script after running 24 hours. I went on the trip the first time starting it without realising this fact. I came home happily and I thought I would think 7 days of data, turn out only one day. You will have to reconnect it every time without your code crashing. Use try/except with a while(1) loop to do some. See my script.

- aggTrade data alone

  Using aggTrade data alone is also doable when you backtest your strategy, but combining it with real time orderbook data will make it much more realistic.

- **Cautious to variance (DeepSeek Wrote this)**

  The variance I first calculated from the first day is the variance of the log-returns $\ln\left(\frac{S_t}{S_{t-1}}\right)$, which is dimensionless and extremely small. When I plugged it directly into the Avellaneda–Stoikov formulas, both the inventory skew and the dynamic spread were essentially zero, so the bot behaved exactly like a constant-spread market maker and always blew up. The correct approach is to convert the log-return variance into a dollar variance by multiplying it by the current mid-price squared: $\sigma^2_{\text{USDT}} = \sigma^2_{\log} \times S_t^2$. The same scaling must be applied to *both* the reservation price skew and the dynamic half-spread. Forgetting to scale the variance, or only scaling one of the two terms, leads to catastrophic losses or a complete failure of inventory control.

- Maker fee

  Be aware of the unit you are using when you consider the effect of maker fee and make sure they are consistent across classes and functions. The first time I ran this I carelessly used 200% fee, which results in at least ten million USDs loss with all set of parameters.